

Adaptive Retrieval for Repository-Level Code Completion

Midterm Progress Report — Group 17

1. Project Overview

Modern code-completion systems rely on retrieval-augmented generation (RAG): when the developer asks the model to fill in a hole, the system first searches the repository for relevant snippets, prepends them to the prompt, and only then calls the language model. RAG works, but indiscriminate retrieval is wasteful. Every completion pays for query construction, an index lookup, and the cost of generating with a much longer prompt. The natural question is whether retrieval can be *conditioned* on need: trigger it only when the model is actually likely to benefit, and skip it otherwise.

CARD (Zhang et al. 2024) is the current state of the art on this question. It trains a small gradient-boosted regressor to predict the edit similarity between the model's zero-shot completion and the ground truth, using only token-level features extracted from the model's own logits. If the predicted similarity is below a threshold, retrieval is invoked; otherwise the zero-shot completion is returned as-is.

Our project reimplements CARD and asks a follow-on question: **does CARD's logit-based confidence signal miss a specific class of failures — confidently-generated identifiers that don't actually exist in the repository?** Confident hallucinations are exactly the case where a logit-based signal cannot help, because the model is wrong but does not know it. Our novel contribution is a **three-tier static-analysis stage** that parses the model's prediction and triggers retrieval when any of three structural checks fire: (1) an identifier in a structurally significant position (a function being called, an object whose attribute is being accessed, the value being subscripted) is not visible in the local scope; (2) a call does not match the callee's known signature; or (3) an import names a symbol that lives in a different module than the one being imported from.

The cascade is asymmetric by design: the static stage can only *add* retrievals to CARD's decisions, never remove them. This bounds the worst-case retrieval rate to the always-retrieve baseline and frames the central research question cleanly — does the additional retrieval budget reduce hallucinations enough to be worthwhile?

Why this matters. Repository-level completion is one of the most widely deployed language-model applications (Copilot, Cursor, JetBrains AI). Every avoided retrieval is saved latency and cost; every avoided hallucination is preserved developer trust. A method that improves the cost-quality trade-off — using only the model's own output and lightweight syntactic analysis — would be architecture-agnostic and easy to deploy.

Research questions

We focus on two questions:

RQ	Question	Hypothesis
RQ1	How does the proposed cascade perform against no-retrieve and always-retrieve baselines on CrossCodeEval-Python, measured by accuracy (Exact Match, Edit Similarity, Identifier-F1) and efficiency (retrieval rate, latency)?	H1. The cascade matches or exceeds always-retrieve on accuracy while performing strictly fewer retrievals.

RQ2	Does the static-analysis cascade reduce the rate of identifier hallucinations compared to vanilla CARD on the same instances?	H2. The cascade lowers the per-instance hallucination rate relative to vanilla CARD, significantly under McNemar's exact test ($p < 0.05$).
------------	---	--

2. Progress So Far

2.1 Background research

We grounded the project in five core references. **CARD** (Zhang et al. 2024) supplies the adaptive-retrieval framework and the per-token feature schema we reimplement. **RepoCoder** (Zhang et al. 2023) and **Repoformer** (Wu et al. 2024) define the iterative-retrieval and selective-retrieval baselines our work is benchmarked against. **CrossCodeEval** (Ding et al. 2023) supplies the evaluation benchmark and the Identifier-F1 metric. **FLARE** (Jiang et al. 2023) provided early evidence that signals derived from the model's own outputs can usefully gate retrieval. To the best of our knowledge, using structural static analysis of the prediction to gate retrieval is novel as an addition to the CARD-style framework.

2.2 Design decisions

- **Asymmetric cascade.** The static-analysis stage can only add retrievals to CARD's decisions, so cascade cost is bounded above by the always-retrieve baseline. This lets us frame RQ2 cleanly as “is the additional retrieval worth it?”
- **Three tiers, structured by what they verify.** Tier 1 (out-of-scope) catches identifiers in structurally significant positions that are not visible in the current file's scope. Tier 2 (signature) catches call sites whose arguments do not match a known repository function's signature. Tier 3 (import origin) catches *from X import Y* statements where Y is actually defined or re-exported elsewhere. The tiers can be independently enabled or disabled, which gives us natural ablations.
- **Tier 1 deliberately ignores the repository-wide symbol table.** A repository-wide symbol table is only as good as the slice of the repo it sees, and on the evaluation benchmark the available slice is partial (chunks selected by BM25, not the entire codebase). Using symbol-table membership to *route* the trigger would penalise real names that happen to live outside the slice. We therefore use a simpler, more conservative rule: an identifier is “out of scope” iff it is not visible in the local file.
- **Hierarchy of structural significance.** Not every unresolved name is a hallucination. A bare identifier in a binary expression might be a parameter we missed, a global, or a fluent style. But an unresolved function being called, an unresolved object being indexed, or an unresolved receiver of an attribute access is much harder to explain away. The analyzer fires only on these structurally significant positions.
- **Multiple generator backends behind one interface.** Because the Estimator depends on per-token logits from the underlying language model, the project supports several backends (HuggingFace, vLLM on GPU, mlx-lm on Apple Silicon, and a deterministic mock for development) all producing the same logit data. This makes calibration portable across deployment environments.
- **Generation caching across configurations.** Most of the configurations being compared share the same zero-shot generations — what differs is what they do afterwards. We cache zero-shot generations on disk keyed by model and prompt, so the headline experiments run from a single shared generation pool rather than regenerating for each configuration.

2.3 Dataset preparation

- **CrossCodeEval-Python** — the primary benchmark, 2,665 instances drawn from real GitHub projects, each with a held-out completion target and a pool of retrievable chunks from the same

repository. Downloaded, parsed, and indexed (per-instance and unioned per-repository).

- **Estimator training data** — sampled from The Stack (a permissively-licensed code corpus), de-duplicated by K-means in feature space, and scored against the language model under evaluation. The current training pool is calibrated for the 1.5B-parameter generator we used for preliminary results; re-calibration for the 7B generator is pending.
- **RepoEval** — supported as a secondary benchmark but not yet exercised; we will draw on it only if the primary results need a second validation table.

2.4 Prototype implementation

The system is feature-complete for the two research questions modulo the 7B-model run. Six configurations are supported end-to-end — the two baselines, vanilla CARD, the cascade, and two extra configurations (static-only and an oracle upper bound) that fall out for free and feed into the analysis.

Config	Stages	Purpose
C1 no-retrieve	Zero-shot only	Lower bound on accuracy (RQ1 baseline).
C2 always-retrieve	BM25 + regenerate every instance	Upper bound on cost (RQ1 baseline).
C3 CARD	Zero-shot → CARD gate → (maybe retrieve)	Vanilla CARD — the RQ2 comparison target.
C4 cascade	Zero-shot → CARD → (if skip) static → (maybe retrieve)	Our contribution. Main RQ1+RQ2 configuration.
C5 static-only	Zero-shot → static gate → (maybe retrieve)	Ablation: how much retrieval does static <i>alone</i> trigger?
C6 oracle	Ex-post pick C1 vs C2 by ground-truth score	Empirical upper bound on any selective policy.

2.5 Experiments conducted

- **Full benchmark run with the 1.5B-parameter generator.** All four core configurations (C1, C2, C3, C4) executed on the complete CrossCodeEval-Python set (2,665 instances). Total wall-clock time approximately six hours, exploiting the shared-generation cache. Results in §4 below.
- **Hallucination-metric calibration.** The C1 and C2 predictions were re-scored under three candidate definitions of “hallucination” (strict significant-position, three-tier union, and an early loose-unresolved rule) to understand how the metric’s definition affects the numbers. This informed the design decision discussed in §3.3.
- **Static-analyzer behavioural evaluation.** Every trigger position the analyzer reasons about — call targets, attribute receivers, subscript values, decorators, raise targets, exception types, class bases, signature mismatches, wrong-origin imports — has been validated against hand-crafted examples covering both positive and negative cases.

3. Current Status

3.1 What is working

- The full pipeline runs end-to-end on the primary benchmark with the 1.5B-parameter generator across all six configurations. Per-instance results are written in a structured format with a stable schema, ready to be consumed by the analysis layer.
- The three-tier static analyzer is implemented with per-tier enable/disable flags. The flags allow each tier's contribution to be measured independently — useful both for the planned ablations and for diagnosing where the cascade's additional retrievals come from.
- The CARD reimplement — the 13-dimensional per-token feature vector, the gradient-boosted regressor, and the threshold gate — is calibrated for the 1.5B-parameter generator with a held-out mean-squared error of approximately 0.044 on the validation pairs.
- The shared generation cache yields near-100% hit rate when re-running downstream configurations, so iteration on the static-stage logic or the CARD threshold is cheap.
- The analysis layer produces the aggregate tables, paired statistical tests (McNemar, paired bootstrap), and the trigger-reason breakdowns needed to answer both research questions; the only missing input is the final 7B-model results.

3.2 What is incomplete

- **Calibration and full run on the 7B-parameter generator.** All results to date use a 1.5B-parameter model. The CARD Estimator is generator-specific (its features are derived from the language model's logits), so the entire training and evaluation cycle has to be re-run on the larger model before the final results are reported. This is the single largest remaining work item.
- **RQ2 statistical test.** The McNemar comparison between vanilla CARD (C3) and the cascade (C4) requires the final hallucination definition to be locked in (see §3.3) and — more importantly — the 7B-model run, because on the 1.5B model CARD saturates and C3/C4 collapse onto the always-retrieve baseline (see §4).
- **Final paper.** Section structure and figure templates are prepared; the empirical sections will be drafted once the 7B numbers are in.

3.3 Plan changes vs. original proposal

- **Scope narrowed from three research questions to two.** An earlier draft included an independent reproduction of CARD's reported numbers as RQ3. Reproducing CARD's exact table requires CodeLlama-7B on RepoEval-line, which is out of reach for our compute budget. We have dropped this third question and rely instead on the internal validation of the Estimator and the controlled comparison structure of RQ1 and RQ2 to support our claims.
- **Hallucination metric refined.** Our first definition was “any unresolved identifier anywhere in the prediction.” On the 1.5B baseline this fires on roughly nine out of every ten predictions, which makes the metric uninformative and would make the cascade look indistinguishable from always-retrieve. We tightened the definition to structurally significant positions only, then extended it into the current three-tier union (out-of-scope, signature mismatch, wrong-origin import). The metric now reflects what the static analyzer can actually *verify* as wrong, rather than what it merely cannot prove right.
- **Local development environment switched to Apple Silicon.** Our original plan assumed access to a GPU cluster throughout. In practice we developed against mlx-lm on a local M-series MacBook and treated GPU access as a finite resource reserved for the headline 7B run. The

deployment-time generator interface is shared, so no code changes are needed when we move from one to the other.

4. Preliminary Results

The numbers below come from a single full run of all four core configurations on the entire CrossCodeEval-Python benchmark (2,665 instances), using a **1.5B-parameter code-completion model**. The headline run with the 7B model is still pending; we present these results as preliminary, and note that the model-size effects they reveal are themselves informative.

Config	Retr. %	EM	ES	IdF1	RSP	Hall.%	Lat. (ms)
C1 no-retrieve	0.0	8.3	27.0	46.1	53.7	78.9	1,255
C2 always-retrieve	100.0	34.4	46.8	62.1	53.5	86.4	2,127
C3 CARD	100.0	35.2	47.5	63.2	51.2	—	3,382
C4 cascade	100.0	34.4	46.8	62.1	50.9	—	3,382

Table 1. Aggregate results on CrossCodeEval-Python ($n = 2,665$) with a 1.5B-parameter generator. EM, ES, IdF1, RSP shown as percentages. RSP (repository symbol precision) is the fraction of identifiers in a prediction that are visible in the local file’s scope. Hall.% is the three-tier hallucination rate (out-of-scope $\vee$ signature mismatch $\vee$ wrong-origin import). The Hall.% values for C3 and C4 are omitted because they were collected under an earlier definition of the metric and will be recomputed together with the 7B run for a clean paired comparison.

Observations

- **Retrieval is decisive on this benchmark.** Moving from no-retrieve to always-retrieve raises Exact Match by a factor of 4.2 (8.3% $\rightarrow$ 34.4%) and Edit Similarity by 73% (27.0% $\rightarrow$ 46.8%). CrossCodeEval was curated to test cross-file dependence, and this is what shows up. The cost is a 70% latency penalty applied to every instance.
- **The CARD gate saturates at 1.5B.** With the default confidence threshold, CARD’s predicted edit similarity falls below the threshold on every single instance, so the gate triggers retrieval 100% of the time. As a consequence vanilla CARD (C3) and the cascade (C4) collapse onto the always-retrieve baseline. This is the central reason the 7B run is essential: a model with sharper logits is needed to give the gate something to discriminate on. A threshold sweep is planned alongside the 7B run.
- **The cascade behaves as designed.** The cascade’s retrieval rate exactly matches vanilla CARD’s, confirming the asymmetric guarantee held in practice: the static stage can only add to CARD’s retrievals, and since CARD’s set is already the full benchmark, the static stage has nothing to add. Accuracy and latency match C3 within floating-point tolerance.
- **The static analyzer exercises all three tiers in practice.** Across the 2,665 predictions, the out-of-scope tier fires on the majority of records (the Hall.% column for the baselines is dominated by Tier 1 fires); the signature and import-origin tiers contribute additional fires whose breakdown will be quantified in the next analysis pass.

Architecture

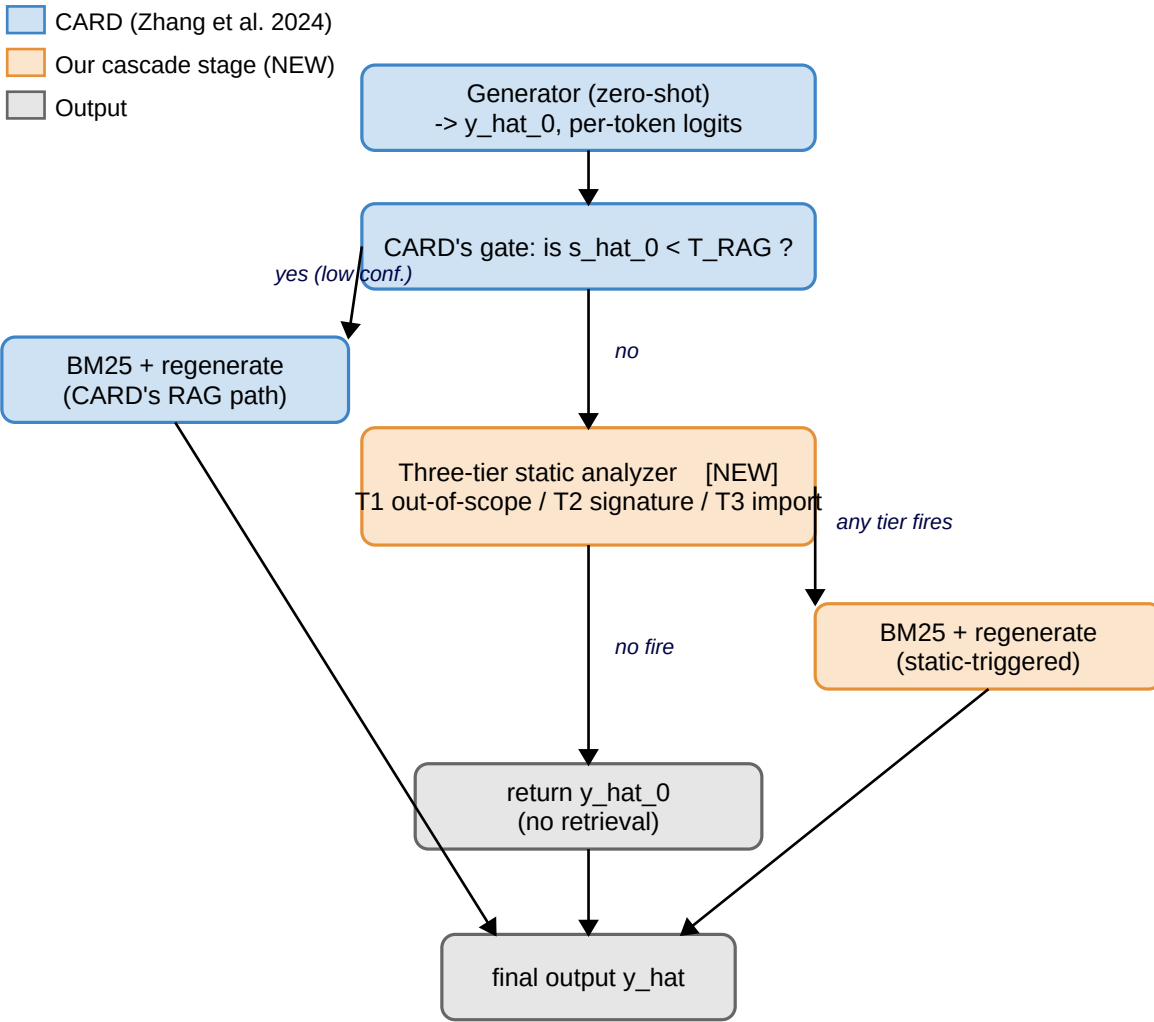


Figure 1. The cascade pipeline. The generator first produces a zero-shot completion. CARD's gate then predicts the completion's edit similarity from the generator's logits; if the predicted similarity is below the threshold, retrieval is triggered. If CARD skips, the three-tier static analyzer parses the prediction and triggers retrieval if any enabled tier fires. Otherwise the zero-shot completion is returned. Blue components are reimplemented from CARD; orange components are our novel contribution.

5. Plan for the Remaining Weeks

The remaining work falls into three coherent phases: finalising the 1.5B baseline under the chosen hallucination metric, calibrating and running on the 7B-parameter generator, and writing up the results. A two-week buffer at the end absorbs any required reruns.

Week	Milestone
Week 5 (now)	Lock the three-tier hallucination definition; re-score all four 1.5B-baseline configurations under the final metric to produce a consistent results table. Secure GPU access for the 7B run (cluster allocation or long-running local backup).
Week 6	Generate Estimator training data using the 7B generator, deduplicate, and fit the regressor. Run a small-scale pilot to confirm the pipeline functions end-to-end with the new generator before committing to the full benchmark.
Week 7	Full evaluation on CrossCodeEval-Python with the 7B generator across all four core configurations. Threshold sweep at three points to characterise the cost-accuracy frontier. Static-only and oracle configurations for the ablations.
Week 8	Final statistical analysis: McNemar (C3 vs C4 on hallucinations), paired bootstrap on accuracy metrics, per-tier contribution breakdown. Production figures: threshold sweep, trigger-mix breakdown, cost-accuracy frontier.
Week 9	Final write-up: draft the empirical sections, discussion, related work, and threats to validity. Internal review at mid-week, polished draft by end-week.
Week 10 (buffer)	Held in reserve for any re-runs prompted by the analysis pass and, if compute allows, a secondary results table on a second benchmark.

Risks

- **Compute access for the 7B run.** The preferred path is the institutional GPU cluster; the verified fallback is overnight execution on a local Apple Silicon machine (estimated 24–36 hours of continuous run time). The fallback works; the cluster is faster.
- **CARD continuing to saturate at 7B.** If predicted similarities at 7B still fall uniformly below the default threshold, the threshold sweep planned for Week 7 will find an operating point that discriminates. If that also fails, the Estimator can be re-fit on training pairs drawn from a benchmark-like distribution rather than the broader code corpus.
- **Hallucination metric too noisy.** Because the metric is reportable per tier, a fall-back is available: report only the signature and import-origin tiers, which are by construction precise (they require concrete evidence of mismatch rather than absence of evidence of definition).